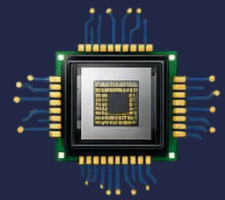
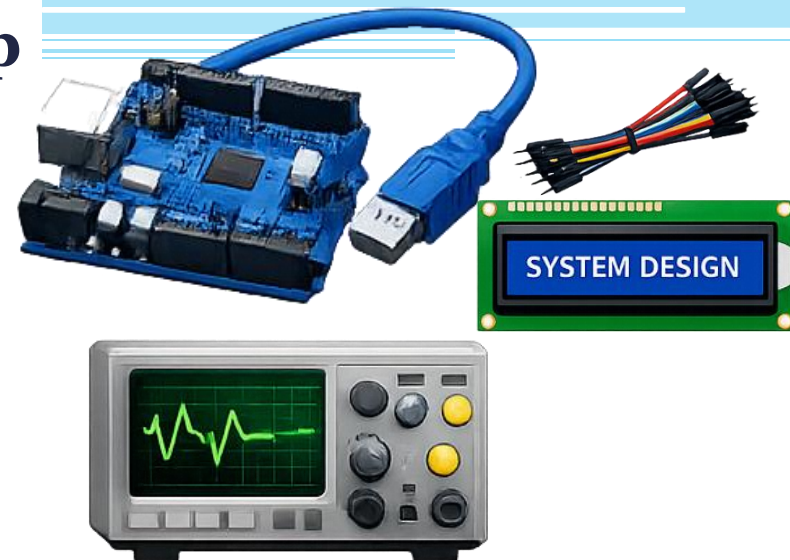


CS-301 Microprocessor- Based System Design



Lecture 27: Accessing On-chip Memory (EEPROM)

Course Teacher: Syeda Ramish Fatima



On-chip EEPROM

- Non-volatile memory
- Byte addressable (512 Bytes)
- Stores configuration & user data
- Used for:
 - Calibration values
 - User settings
 - Counters / logs

On-chip EEPROM

- EEPROM in AVR (like ATmega162) can be written in four ways:
 - **SPI (ISP – In-System Programming)**
 - External programmer (e.g., **Khazama AVR Programmer**)
 - Uses SPI pins
 - Programs EEPROM before execution
 - **JTAG Programming**
 - Uses JTAG interface
 - Can program and debug
 - Also external, like SPI

On-chip EEPROM

- EEPROM in AVR (like ATmega162) can be written in four ways:
 - **Runtime EEPROM access**
 - Program writes EEPROM **during execution**
 - Uses EEAR, EEDR, EECR
 - **Bootloader**
 - Data comes via UART/USB
 - Bootloader (in Flash) runs
 - Then writes EEPROM using EECR

On-chip EEPROM Registers

1. EEAR (EEPROM Address Register)

Holds address (0–511)

2. EEDR (EEPROM Data Register)

Holds data to be written or data read

3. EECR (EEPROM Control Register)

Controls EEPROM operations.



On-chip EEPROM Registers

these are EECR bits

Bit	Name	Function
2	EEMWE	Master Write Enable
1	EEWE	Write Enable
0	EERE	Read Enable 1 to read

EEPROM Write Sequence

- Wait until previous write completes
 - (EWE = 0)
- Load address into **EEAR**
- Load data into **EEDR**
- Set **EEMWE = 1**
- Within 4 cycles Because it is a two-step protection latch system to prevent accidental EEPROM writes.
 - Set **EWE = 1**



EEPROM Read Sequence

- Wait until $EEWE = 0$
- Load address into $EEAR$
- Set $EERE = 1$
- Read data from $EEDR$



A simple application to write and read from on-chip EEPROM

```
/*
 * EEPROM.c
 *
 * /

#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>

// EEPROM Write Function
void EEPROM_write(unsigned int address, unsigned char data)
{
    // Wait until previous write completes
    while (EECR & (1<<EWE)); 1 << EWE shifts binary 00000001 left by EWE positions

    // Set address and data
    EEAR = address;
    EEDR = data;
```

A simple application to write and read from on-chip EEPROM

```
// Enable write
EECR |= (1<<EEMWE);

// Start write
EECR |= (1<<EEWE);
}

// EEPROM Read Function
unsigned char EEPROM_read(unsigned int address)
{
    // Wait until previous write completes
    while (EECR & (1<<EEWE));

    // Set address
    EEAR = address;

    // Start read
    EECR |= (1<<EERE);

    // Return data
    return EEDR;
}
```

A simple application to write and read from on-chip EEPROM

```
int main(void)
{
    unsigned char data;
    char addr=0x20;
    // Set PORTB as output
    DDRB = 0xFF;

    // Write value 0x55 to EEPROM address 0x20
    EEPROM_write(0x20, 0x55);
    EEPROM_write(0x21, 0xAA);
    EEPROM_write(0x22, 0x01);
    EEPROM_write(0x23, 0x88);
```

•

A simple application to write and read from on-chip EEPROM

```
while(1)
{
// Read back from EEPROM
if(addr==0x24)
addr=0x20;
data = EEPROM_read(addr);
// Output to PORTB
PORTB = data;
addr++;
_delay_ms(1000);
}
}
```

•